

Linear time algorithm for the longest common repeat problem

Inbok Lee^{a,b,1}, Costas S. Iliopoulos^b, Kunsoo Park^{a,*}

^a School of Computer Science & Engineering, Seoul National University, Seoul, Republic of Korea

^b Department of Computer Science, King's College London, London, United Kingdom

Available online 16 May 2006

Abstract

Given a set of strings $U = \{T_1, T_2, \dots, T_\ell\}$, the longest common repeat problem is to find the longest common substring that appears at least twice in each string of U . We also consider reversed and reverse-complemented repeats as well as normal repeats. We present a linear time algorithm for the longest common repeat problem.

© 2006 Elsevier B.V. All rights reserved.

Keywords: String algorithm; Repeat; Generalized suffix tree

1. Introduction

Repetitive or periodic strings have a great importance in a variety of applications including computational molecular biology, data mining, data compression, and computer-assisted music analysis. For example, it is assumed that repetitive substrings in a biological sequence have important meanings and functions [1]. Finding common substrings in a set of strings is also important. For example, motifs or short strings common to protein sequences are assumed to represent a specific property of the sequences [3].

In this paper we want to find common repetitive substrings in a set of strings. We especially focus on finding the longest common repeat in a set since the number of the common repeats in a set can be quite large. We also consider reversed and reverse-complemented strings in finding repeats. Formally we define our problem as follows.

Let T be a string over an alphabet Σ . We assume $\Sigma = \{A, C, G, T\}$ or $\Sigma = \{A, C, G, U\}$ since a major application of the problem is computational molecular biology. $T[i]$ denotes the i th character of T . $T[i..j]$ is the substring $T[i]T[i+1]\dots T[j]$ of T . The *left character* of a suffix $T[i..|T|]$ means $T[i-1]$. T^R denotes the *reverse string* of T where $|T^R| = |T|$ and $T^R[i] = T[|T| - i + 1]$ for $1 \leq i \leq |T|$. T^{RC} denotes the *reverse-complemented string* of T where $|T^{RC}| = |T|$ and, $T^{RC}[i]$ and $T[|T| - i + 1]$ form a Watson–Crick pair ($A \equiv (T \text{ or } U)$ and $C \equiv G$) for $1 \leq i \leq |T|$.

A *repeat* of T is a substring of T which appears at least twice in T . There are three kinds of repeats.

* Corresponding author.

E-mail addresses: iblee@theory.snu.ac.kr (I. Lee), csi@dcs.kcl.ac.uk (C.S. Iliopoulos), kpark@theory.snu.ac.kr (K. Park).

¹ This work was supported by the Korea Research Foundation Grant funded by Korean Government (MOEHRD, Basic Research Promotion Fund) M01-2004-000-20344-0.

- **Normal repeat**: A string p is called a *normal repeat* of T if $p = T[i..i + |p| - 1]$ and $p = T[i'..i' + |p| - 1]$ for $i \neq i'$.
- **Reversed repeat**: A string p is called a *reversed repeat* of T if $p = T[i..i + |p| - 1]$ and $p^R = T[i'..i' + |p| - 1]$.
- **Reverse-complemented repeat**: A string p is called a *reverse-complemented repeat* if $p = T[i..i + |p| - 1]$ and $p^{RC} = T[i'..i' + |p| - 1]$.

There are two reasons why we consider reversed and reverse-complemented repeats: (i) We don't know the directions of the strings in advance. (ii) In some situations, reversed and reverse-complemented repeats play an important role. For example, RNA secondary structures are determined by reverse-complemented repeats.

The longest common repeat problem can be defined as follows.

Problem 1. Given a set of strings $U = \{T_1, T_2, \dots, T_\ell\}$, the (k, ℓ) longest common repeat problem is to find the longest repeat (normal, reversed or reverse-complemented) which are common to k strings in U for $1 \leq k \leq \ell$.

For finding the longest normal repeat in a text T , Karp, Miller, and Rosenberg first proposed $O(|T| \log |T|)$ time algorithm [8]. However, it is an easy application of the suffix tree [4,10,13] to find it in $O(|T|)$ time.

For approximate normal repeats, Landau and Schmidt gave an $O(k|T| \log k \log |T|)$ time algorithm for finding approximate squares where the allowed edit distance is at most k [9]. Schmidt also gave an $O(|T|^2 \log |T|)$ time algorithm for finding approximate tandem or non-tandem repeats [12].

The longest common repeat problem resembles the longest common substring problem. The difference is that the common substring should appear at least twice in each sequence in the longest common repeat problem. For the longest common substring problem with a set of strings $\{T_1, T_2, \dots, T_\ell\}$, Hui showed an $O(\sum_{i=1}^{\ell} |T_i|)$ time algorithm [7]. As far as we know, our algorithm is the first one that solves the longest common repeat problem.

2. Preliminaries

A *generalized suffix tree* stores all the suffixes of a set of strings just as a suffix tree stores all the suffixes of a string. It is easy to extend the suffix tree construction algorithm [13] to building a generalized suffix tree [5, p. 116]. Fig. 1 is an example of the generalized suffix tree for $T_1 = AACTG$ and $T_2 = ACTGCTG$. We use a special character $\$$ which is not in Σ to denote the end of a string. Each leaf node has an ID representing the original string where the suffix came. Identical suffixes of two or more strings are considered as different ones. In this example, T_1 and T_2 share three identical suffixes CTG , TG , and G . Each of these suffixes has two leaves with different IDs.

From now on, let $ST(T)$ denote the suffix tree of T and $GST(T_1..T_\ell)$ denote the generalized suffix tree of $T_1, T_2, \dots, T_\ell$. Let $L(v)$ denote the string obtained by concatenating the edge labels on the path from the root to a node v in a suffix tree or a generalized suffix tree.

We define *corresponding nodes* between $ST(T_i)$ and $GST(T_1..T_\ell)$ ($1 \leq i \leq \ell$).

Definition 1. The *corresponding node* of an internal node v in $ST(T_i)$ ($1 \leq i \leq \ell$) is a node v' in $GST(T_1..T_\ell)$ such that $L(v) = L(v')$.

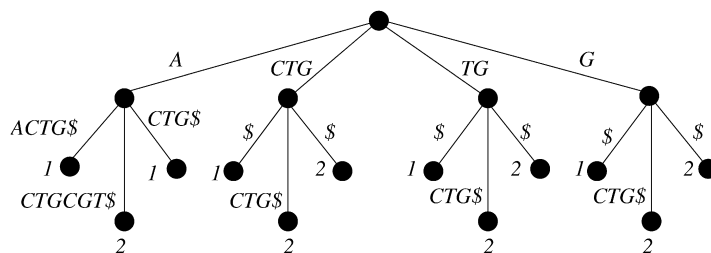


Fig. 1. The generalized suffix tree for $T_1 = AACTG$ and $T_2 = ACTGCTG$.

- *Step 1*: Create a new string T'_i for each $1 \leq i \leq \ell$ to consider reversed and reverse-complemented repeats.
- *Step 2*: Build $ST(T'_i)$ for each $1 \leq i \leq \ell$. Also, build $GST(T'_1..T'_\ell)$.
- *Step 3*: Find supermaximal repeats of T'_i for each i in $GST(T'_1..T'_\ell)$.
- *Step 4*: Modify $GST(T'_1..T'_\ell)$ and build the generalized suffix tree of the supermaximal repeats.
- *Step 5*: Find the longest common repeat among the supermaximal repeats using the generalized suffix tree made in Step 4.

The hard part of the algorithm is Step 4, which changes $GST(T'_1..T'_\ell)$ into the generalized suffix tree of the supermaximal repeats in linear time.

Step 1: We first modify each string in U to consider the reversed and reverse-complemented repeats. For each $i = 1, 2, \dots, \ell$, we create a new string $T'_i = T_i \% T_i^R \# T_i^{RC}$, where $\%$ and $\#$ are special characters which are not in Σ . Normal repeats of T'_i include normal, reversed, and reverse-complemented repeats of T_i .

Step 2: We build the suffix trees and the generalized suffix tree. For each $i = 1, 2, \dots, \ell$, we build $ST(T'_i)$ with the following modification. When we create an internal node v , we store an additional information (j, j') at v . It means that v is the lowest common ancestor (LCA) of two leaves representing $T'_i[j..|T'_i|]$ and $T'_i[j'..|T'_i|]$, respectively. If there are more than two leaves in the subtree rooted at v , arbitrary two leaves can be chosen. See Fig. 4. (We can store $(3, 1)$ instead of $(3, 2)$ at the second internal node.) This modification does not change the time and space complexities. We also build $GST(T'_1..T'_\ell)$. This procedure runs in $O(\sum_{i=1}^{\ell} |T'_i|)$ time and space.

Step 3: We find supermaximal repeats of each string in U . For each $i = 1, 2, \dots, \ell$, we find supermaximal repeats of T'_i using $ST(T'_i)$ [5, pp. 143–148]. Now we have a set of nodes $M_i = \{v | v \text{ is an internal node of } ST(T'_i) \text{ and } L(v) \text{ is a supermaximal repeat of } T'_i\}$. We compute the set of corresponding nodes $V_i = \{v' | v' \text{ is an internal node of } GST(T'_1..T'_\ell) \text{ and it is the corresponding node of } v \in M_i\}$. To do so, we use the information obtained during the construction of $ST(T'_i)$. Fig. 5 illustrates the idea. We read the information (j, j') at v computed in Step 2. Then we compute the LCA v' of two leaves in $GST(T'_1..T'_\ell)$ representing $T'_i[j..|T'_i|]$ and $T'_i[j'..|T'_i|]$, respectively. It is easy to show that v' is the corresponding node of v . After $O(\sum_{i=1}^{\ell} |T'_i|)$ -time LCA preprocessing, finding a corresponding node takes constant time [2,6,11]. The total time complexity is $O(\sum_{i=1}^{\ell} |T'_i|)$.

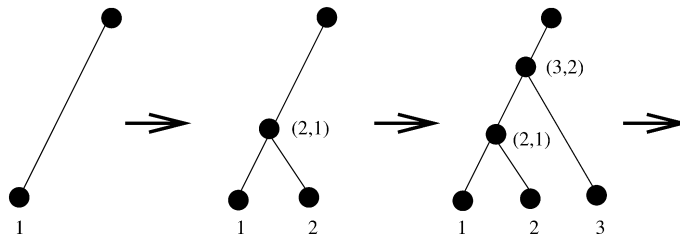


Fig. 4. Modification of the suffix tree construction.

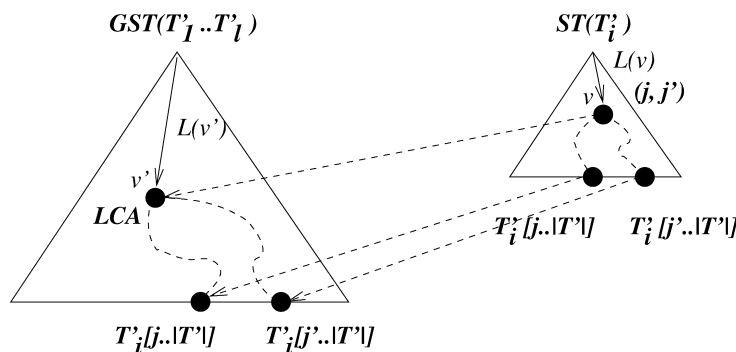


Fig. 5. Finding the corresponding node of v .

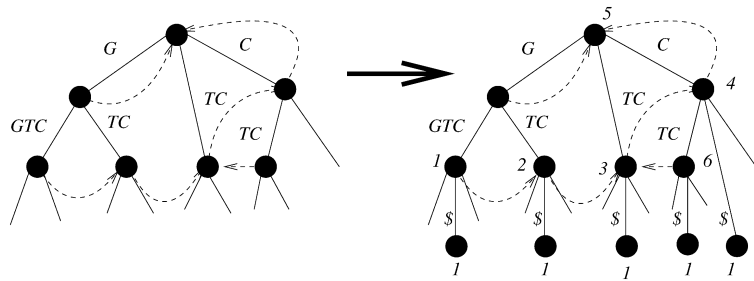


Fig. 6. Inserting the suffixes of $GGTC$ and CTC into $GST(T'_1..T'_\ell)$.

Step 4: Now we explain the hard part of the algorithm, modifying $GST(T'_1..T'_\ell)$ into the generalized suffix tree of the supermaximal repeats in linear time. At this point we have sets V_i 's, where $L(v)$ for each $v \in V_i$ ($1 \leq i \leq \ell$) is a supermaximal repeat of T'_i .

The outline of Step 4 is as follows.

- (1) For each supermaximal repeat S , insert the suffixes of $S\$$ into $GST(T'_1..T'_\ell)$.
- (2) Identify the nodes of the current tree which should be included in the generalized suffix tree of the supermaximal repeats.
- (3) Remove the unnecessary nodes and edges of the tree.

We first insert the suffixes of the supermaximal repeats into $GST(T'_1..T'_\ell)$. For each $i = 1, 2, \dots, \ell$, we traverse $GST(T'_1..T'_\ell)$ from each element of V_i to the root node following the suffix links. Whenever we meet a node v previously unvisited during the traversal, we create a new leaf node with ID i that is a child of node v with label $\$$. These new leaves are stored in a set $N_i = \{v | v \text{ is a leaf node of the tree and } L(v) \text{ is a suffix of a supermaximal repeat of } T'_i \text{ followed by } \$\}$. We stop the traversal when we meet a previously visited node and move to the next element of V_i . Fig. 6 illustrates the idea. Suppose T'_1 has two supermaximal repeats, $GGTC$ and CTC . First we handle $GGTC$. The visiting order is $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$. After visiting node 5, which is the root node, we are done with $GGTC$ and we handle CTC . After visiting node 6, we visit node 3 again, following the suffix link. Then we are done with CTC . This procedure runs in $O(|T'_i|)$ time and $|N_i| = O(|T'_i|)$.

Now we identify the nodes of the tree which should be included in the generalized suffix tree of the supermaximal repeats. To do so, for each $i = 1, 2, \dots, \ell$, we traverse the tree from each element of N_i to the root node upward and mark the nodes on the path. We stop the traversal if we meet a marked node and go on to process the next element of N_i . After this procedure, the marked nodes consist of the nodes which should be included in the generalized suffix tree of supermaximal repeats and some internal nodes which have only one child if we remove the unmarked nodes from the tree. (The unmarked nodes should not be included in the generalized suffix tree of the supermaximal repeats.)

Finally, we remove the unnecessary nodes and edges which should not be in the generalized suffix tree of the supermaximal repeats. We traverse the tree from original leaves (not the new leaves created in Step 4) to the root node upward and delete the nodes and edges on the path if they are unmarked. We move to the next original leaf of the tree if we meet a marked node. After this procedure, the resulting tree may contain marked internal nodes which have only one child. We find these nodes and remove them, merging the edges. Now we have the generalized suffix tree of the supermaximal repeats. Step 4 runs in $O(\sum_{i=1}^{\ell} |T'_i|)$ time.

Step 5: The remaining problem is to find the longest common substring among k supermaximal repeats with distinct IDs in the generalized suffix tree of the supermaximal repeats. Unlike the longest common substring problem, two or more supermaximal repeats can have the same ID here.

Still we can use Hui's algorithm for the longest common substring problem in this case, because it solves a rather general problem [7]. The problem is that each leaf of the tree has a color (an ID in our problem) and that we want to find the deepest node (in the length of $L(v)$) whose subtree has leaves with at least k colors. Note that if the tree is a generalized suffix tree of strings and we use color i for the suffixes of string i , then the problem becomes the longest common substring problem. The number of distinct colors in a subtree of an internal node can be obtained by subtracting the number of duplicate colors from the number of leaves in that subtree. Since we can compute the

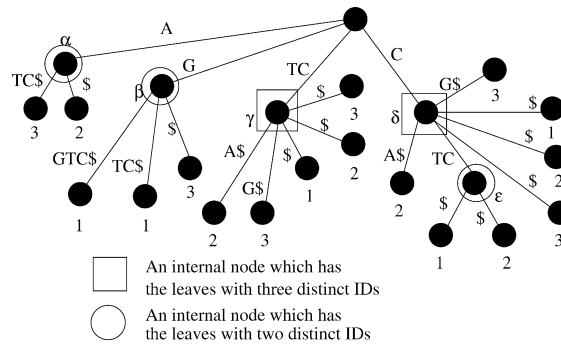


Fig. 7. An example of the longest common repeat problem.

latter easily by the bottom-up traversal, now the problem is to calculate the former efficiently. The key idea of Hui's algorithm is to use the LCA query again. First, we make a list of leaves for each color i , in the order of the depth-first search. Then we compute the LCA for each consecutive pair of leaves in each list. If an internal node v is the LCA of two consecutive leaves with color i , it means that color i is duplicated once in the subtree of v . For each internal node, we count the number of times when the node is the LCA for some pair of consecutive leaves with the same color, which is the number of duplicated colors in the subtree of the node. For a more detailed account we refer the reader to [7] and [5, pp. 127–129, 205–207]. Fig. 7 is an example of finding longest common substring. Suppose T'_1 has two supermaximal repeats $GGTC$ and CTC , T'_2 has two supermaximal repeats CTC and TCA , and T'_3 has two supermaximal repeats TCG and ATC . For each internal node of the generalized suffix tree of the supermaximal repeats, we compute the number of different IDs in its subtrees. The internal nodes with rectangles (nodes γ and δ) have leaves with three different IDs in their subtrees. The internal nodes with circles (nodes α , β , and ϵ) have leaves with two different IDs in their subtrees. For the $(3, 3)$ longest common repeat problem, we compare the lengths of $L(\gamma) = TC$ and $L(\delta) = C$. The answer is TC . For the $(2, 3)$ longest common repeat problem, the answer is $L(\epsilon) = CTC$. It runs in $O(\sum_{i=1}^{\ell} |T'_i|)$ time, reporting the answer of (k, ℓ) longest common repeat problem for all $1 \leq k \leq \ell$.

Theorem 1. The (k, ℓ) longest common repeat problem can be solved in $O(\sum_{i=1}^{\ell} |T_i|)$ time and space for all $1 \leq k \leq \ell$.

Proof. We showed that all the steps run in $O(\sum_{i=1}^{\ell} |T'_i|)$ time. Since the size of $GST(T'_1..T'_\ell)$ is $O(\sum_{i=1}^{\ell} |T'_i|)$ and each string T'_i has a suffix tree whose size is $O(|T'_i|)$, the space complexity is also $O(\sum_{i=1}^{\ell} |T'_i|)$. Since $|T'_i| = O(|T_i|)$, we get the theorem. $\square$

4. Conclusion

We have defined the longest common repeat problem and presented a linear time algorithm for the problem, allowing reversed and reverse-complemented repeats. A remaining work is to devise a space-efficient algorithm for the longest common repeat problem without using the suffix tree. Another possibility is the longest common approximate repeat problem which incorporates insertions, deletions and substitutions.

References

- [1] A. Apostolico, F. Gong, S. Lonardi, Verbumculus and the discovery of unusual words, *Journal of Computer Science and Technology* 19 (1) (2003) 22–41.
- [2] M.A. Bender, M. Farach-Colton, The LCA problem revisited, in: *Proceedings of the Fourth Latin American Symposium*, 2000, pp. 88–94.
- [3] L. Falquet, M. Pagini, P. Bucher, N. Hulo, C.J. Sigrist, K. Hofmann, A. Bairoch, The PROSITE database, its status in 2002, *Nucleic Acids Research* 30 (2002) 235–238.
- [4] M. Farach-Colton, P. Ferragina, S. Muthukrishnan, On the sorting-complexity of suffix tree construction, *Journal of the ACM* 47 (6) (2000) 987–1011.

- [5] D. Gusfield, *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*, Cambridge University Press, Cambridge, 1997.
- [6] D. Harel, R.E. Tarjan, Fast algorithms for finding nearest common ancestor, *SIAM Journal on Computing* 13 (2) (1984) 338–355.
- [7] L.C.K. Hui, Color set size problem with applications to string matching, in: *Proceedings of the Third Annual Symposium on Combinatorial Pattern Matching*, Springer-Verlag, Berlin, 1992, pp. 230–243.
- [8] R.M. Karp, R.E. Miller, A.L. Rosenberg, Rapid identification of repeated patterns in strings, trees and arrays, in: *Proceedings of the Fourth Annual ACM Symposium on Theory of Computing*, 1972, pp. 125–136.
- [9] G.M. Landau, J.P. Schmidt, An algorithm for approximate tandem repeats, in: *Proceedings of the Fourth Combinatorial Pattern Matching*, 1993, pp. 120–133.
- [10] E.M. McCreight, A space-economical suffix tree construction algorithm, *Journal of the ACM* 23 (2) (April 1976) 262–272.
- [11] B. Schieber, U. Vishkin, On finding lowest common ancestors: Simplification and parallelization, *SIAM Journal on Computing* 17 (1988) 1253–1262.
- [12] J.P. Schmidt, All highest scoring paths in weighted grid graphs and its application to finding all approximate repeats in strings, *SIAM Journal on Computing* 27 (4) (1998) 972–992.
- [13] E. Ukkonen, On-line construction of suffix trees, *Algorithmica* 14 (1995) 249–260.